

Lab 9: XiangShan Micro-Architecture

In this lab, you will use UCAgent, an AI-powered automated hardware verification agent based on large language models, to explore agile chip development and unit test verification for the XiangShan project.

By the end of this lab, you will be able to:

1. Install and configure the agile verification infrastructure (Verilator, Picker, UCAgent)
2. Export an RTL design to a Python package using Picker
3. Automate test generation and execution using OpenCode via the MCP protocol
4. Analyze the generated test reports to identify hardware bugs

Default toolchain configuration:

- **Picker:** Converts RTL design verification modules (`.v/.scala/.sv`) into dynamic libraries with high-level language programming interfaces (Python, C++, etc.).
- **Toffee:** A Python-based hardware verification framework.
- **UCAgent:** Driven by Large Language Models (LLMs) to complete requirement understanding, test generation, execution, and report generation.

Note: After modifying any environment variables or installing tools, ensure you reload your terminal.
The Picker version used in this lab requires Verilator version ≥ 5.020 .

Step 1 - Infrastructure Setup

Background You must install Verilator, UCAgent, and Picker to build the testing environment.

```
# 1. Install Verilator (>= 5.020)
sudo apt-get install git help2man perl python3 make g++ libgz libfl2 libfl-dev
zlibc zlib1g zlib1g-dev
git clone https://github.com/verilator/verilator
cd verilator
git checkout v5.020
autoconf && ./configure && make && sudo make install
verilator --version

# 2. Install UCAgent
pip3 install git+https://git@github.com/XS-MLVP/UCAgent@main
ucagent --help

# 3. Install Picker
sudo apt install cmake build-essential
pip3 install swig
git clone https://github.com/XS-MLVP/picker.git
cd picker
make init && make && sudo -E make install
```

Step 2 - AI Agent Configuration

Background UCAgent collaborates with LLM tools via the MCP protocol. We will use **OpenCode** as the primary agent for this lab.

(Note: Other tools supporting the MCP protocol, such as Claude Code, can also be used to drive the verification, but you will need to explore their specific installation and MCP configuration independently).

```
# Install OpenCode CLI (requires a Node.js environment)
sudo npm install -g @opencode/opencode
```

Configure the MCP server for OpenCode by editing its settings file (e.g., `${workspace}/opencode.json` or `~/.config/opencode/opencode.json`):

Note: Make sure your OpenCode version is at least v1.14.28 if you use DeepSeek-V4.

```
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "unitytest": {
      "type": "remote",
      "url": "http://localhost:5000/mcp",
      "timeout": 10000
    }
  }
}
```

Step 3 - DUT Preparation and Export

Background We will use a 64-bit adder with an intentionally injected bug (output bit width modified to 63 bits) to demonstrate width-related defects. You need to create the RTL file, write a specification, and export it as a Python module so Toffee can interface with it.

Create the working directory:

```
mkdir -p Adder
```

Create `Adder/Adder.v`:

```
// A verilog 64-bit full adder with carry in and carry out
module Adder #(
    parameter WIDTH = 64
) (
    input [WIDTH-1:0] a,
```

```

    input [WIDTH-1:0] b,
    input cin,
    output [WIDTH-2:0] sum, // Intentional bug injected here
    output cout
);

assign {cout, sum} = a + b + cin;

endmodule

```

Create `Adder/README.md`:

```

### Adder 64 位加法器
输入 a, b, cin 输出 sum, cout
实现 sum = a + b + cin
cin 是进位输入
cout 是进位输出

### 验证目标
只要验证加法相关的功能，其他验证（例如波形、接口等）不需要出现。

### bug 分析
在 bug 分析时，请参考源码：examples/MyAdder/Adder.v。

### 其他
所有的文档和注释都用中文编写。

```

Export the RTL to a Python module using Picker, and copy the specifications into the output directory:

```

picker export Adder/Adder.v --rw 1 --sname Adder --tdir output/ -c -w
output/Adder/Adder.fst
cp Adder/README.md output/Adder/README.md

```

Step 4 - AI-Driven Verification

Background You will launch the UCAgent MCP server to provide context and tools, then use OpenCode to drive the verification process based on the README specifications. The UCAgent TUI interface allows you to monitor the pipeline.

Open a terminal and start the UCAgent MCP server:

```

ucagent output/ Adder -s -hm --tui --mcp-server-no-file-tools --no-embed-tools

```

(Hint: You can skip certain stages in the TUI interface by executing commands like `skip_stage 7` to prevent excessive API usage. Skipping stages 7 through 27 is recommended).

Open a **second terminal**, navigate to the `output` directory, and start OpenCode:

```
cd output
opencode
```

Input the following prompt into OpenCode (and agree to any tool/read/write requests):

请通过MCP的工具 RoleInfo 获取你的角色信息和基本指导，然后完成任务。请使用工具 ReadTextFile 读取文件。你需要在当前工作目录进行文件操作，不要超出该目录。

Question 1: Submit a screenshot showing the UCAgent TUI interface running successfully during the ADD module verification. You must run the `whoami` command in your terminal and include the output in the screenshot.

Question 2: After the test generation and execution are complete, locate the generated HTML test report in `output/uc_test_report/index.html`. Explain exactly where and what the error is in the ADD module based on the verification results.

Question 3: Summarize the workflow. What specific tasks did UCAgent automate during this lab?

References

- [UCAgent Repository](#)
- [Picker Repository](#)
- [Verilator Installation](#)